

CSC 548 HW2 Report

Unity ID: fclin

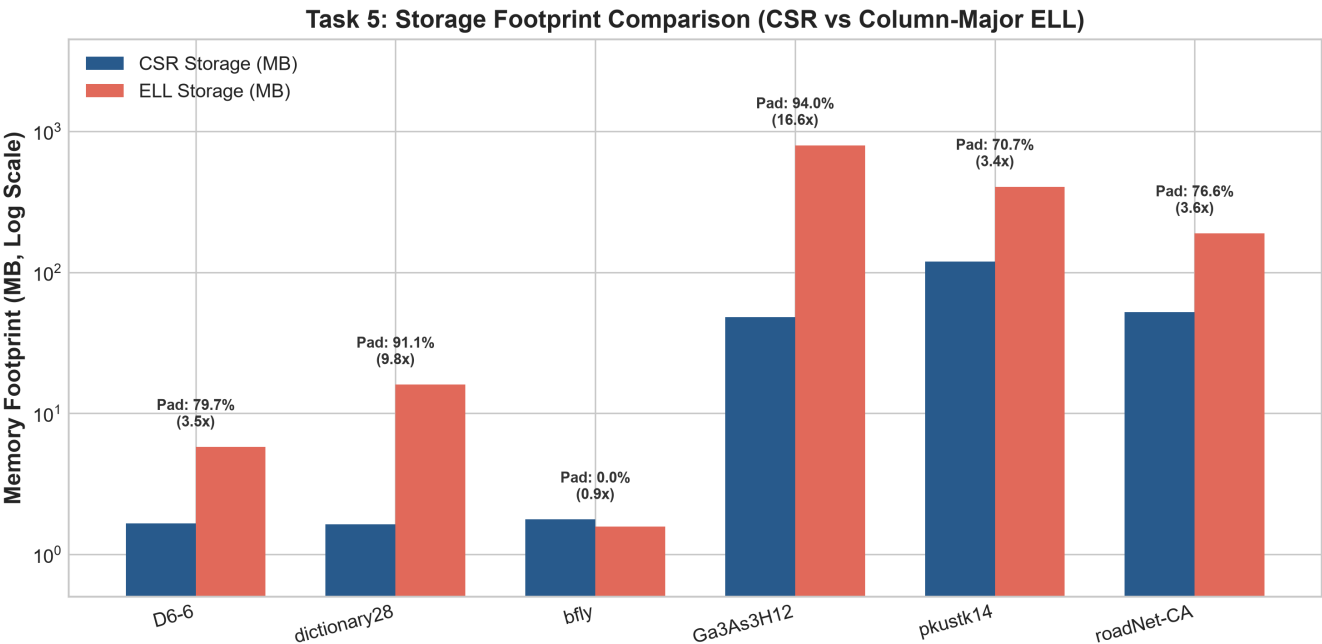
System: NC State ARC Cluster (Dual-Socket Intel Xeon Silver 4110 @ 2.10 GHz, 16 cores, DDR4-2400, Peak Compute: 537.6 GFLOP/s, Peak STREAM Bandwidth: ~200 GB/s)

5. Format Comparison: CSR vs. ELL Memory

Storage Consumption Table

Matrix	Dimensions ($m \times n$)	NNZ	Max K	Avg μ	CSR (MB)	ELL (MB)	Zero Padding	ELL/CSR Ratio
D6-6	$120,576 \times 23,740$	146,880	6	1.2	1.66 MB	5.79 MB	79.7%	3.49×
dictionary28	$52,652 \times 52,652$	178,076	38	3.4	1.64 MB	16.01 MB	91.1%	9.79×
bfly	$49,152 \times 49,152$	196,608	4	4.0	1.77 MB	1.57 MB	0.0%	0.89×
Ga3As3H12	$61,349 \times 61,349$	5,970,947	1,622	97.3	48.01 MB	796.06 MB	94.0%	16.58×
pkustk14	$151,926 \times 151,926$	14,836,504	333	97.7	119.30 MB	404.73 MB	70.7%	3.39×
roadNet-CA	$1,971,281 \times 1,971,281$	5,533,214	12	2.8	52.15 MB	189.24 MB	76.6%	3.63×

Supporting Plot:



Key Findings

ELL wastes memory on matrices in which the average nonzero count per row is much lower than the max. This causes ELL to allocate a large amount of slots per row, which are then padded mostly with empty entries. This can be seen with **D6-6**, **dictionary28**, **Ga3As3H12**, **pkustk14**, **roadNet-CA**, with **Ga3As3H12** being the worst offender.

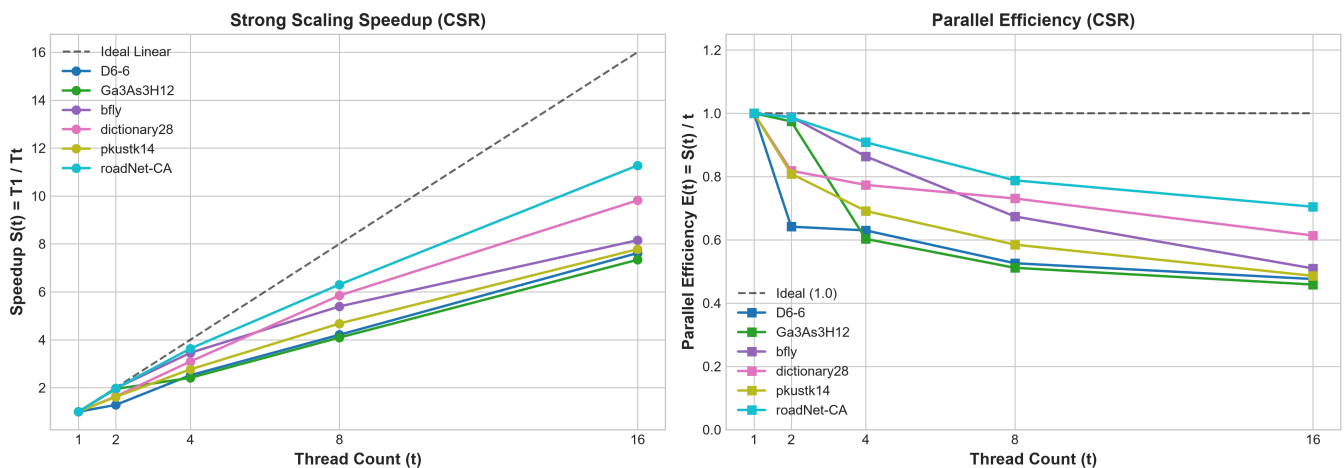
6. Performance, Scaling & Roofline

A. Thread Strong Scaling & Efficiency

$S(t) = \frac{T_1}{T_t}$, $E(t) = \frac{S(t)}{t}$ measured with OpenMP CSR (**static** row partitioning) on 1 to 16 threads:

Matrix	T_1 (ms)	$t = 2$ (S, E))	$t = 4$ (S, E))	$t = 8$ (S, E))	$t = 16$ (S, E))	Max Speedup
D6-6	0.549	0.427 ms (1.28×, 0.64)	0.218 ms (2.52×, 0.63)	0.130 ms (4.21×, 0.53)	0.072 ms (7.62×, 0.48)	7.62×
dictionary28	0.655	0.400 ms (1.64×, 0.82)	0.212 ms (3.09×, 0.77)	0.112 ms (5.85×, 0.73)	0.067 ms (9.82×, 0.61)	9.82×
bfly	0.271	0.137 ms (1.98×, 0.99)	0.079 ms (3.46×, 0.86)	0.050 ms (5.39×, 0.67)	0.033 ms (8.15×, 0.51)	8.15×
Ga3As3H12	7.802	4.004 ms (1.95×, 0.97)	3.235 ms (2.41×, 0.60)	1.906 ms (4.09×, 0.51)	1.063 ms (7.34×, 0.46)	7.34×
pkustk14	18.545	11.466 ms (1.62×, 0.81)	6.707 ms (2.77×, 0.69)	3.964 ms (4.68×, 0.58)	2.384 ms (7.78×, 0.49)	7.78×
roadNet-CA	17.234	8.731 ms (1.97×, 0.99)	4.743 ms (3.63×, 0.91)	2.734 ms (6.30×, 0.79)	1.529 ms (11.27×, 0.70)	11.27×

Supporting Plot:

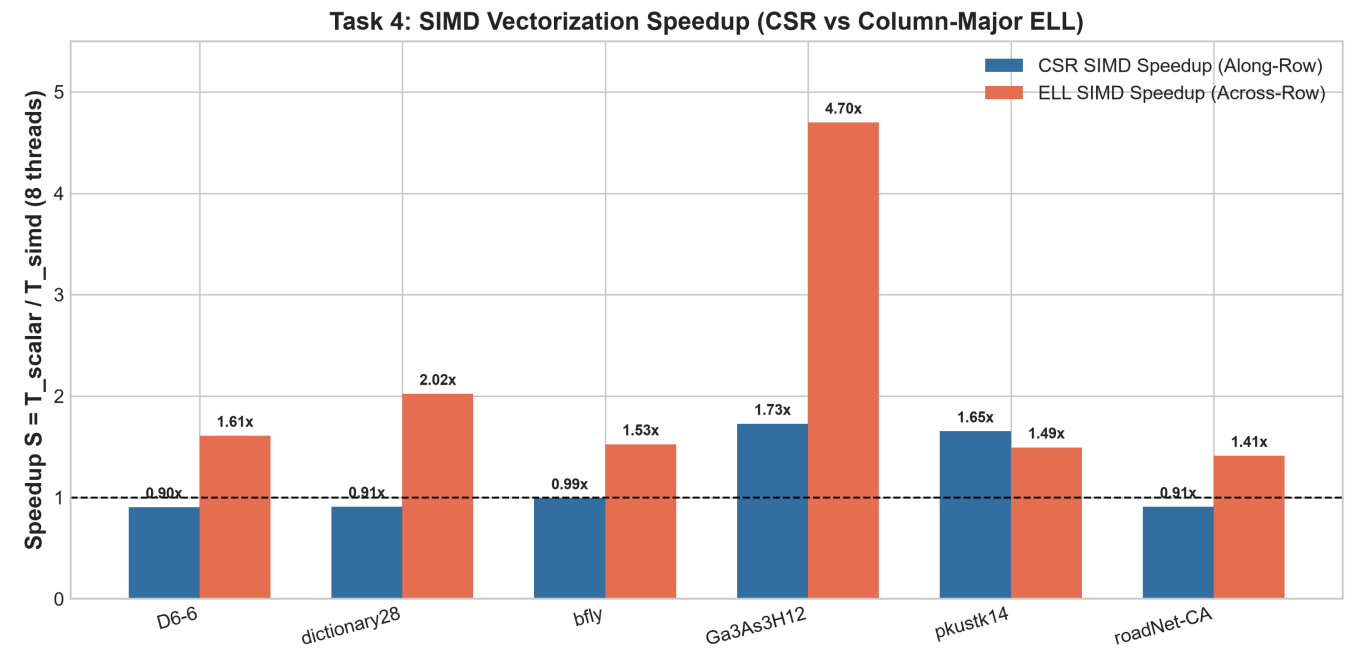


B. SIMD Vectorization Speedup

Comparing Scalar vs. AVX2 SIMD at 8 threads ($S = T_{\text{scalar}}/T_{\text{simd}}$):

Matrix	CSR Scal	CSR SIMD	CSR Speedup	ELL Scal	ELL SIMD	ELL Speedup	Winner
D6-6	0.131 ms	0.145 ms	0.90× (Slowdown)	0.093 ms	0.058 ms	1.61×	ELL SIMD (2.5× faster)
dictionary28	0.112 ms	0.123 ms	0.91× (Slowdown)	0.360 ms	0.178 ms	2.02×	CSR Scalar (1.6× faster)
bfly	0.050 ms	0.050 ms	0.99× (Neutral)	0.040 ms	0.026 ms	1.53×	ELL SIMD (1.9× faster)
Ga3As3H12	1.911 ms	1.107 ms	1.73×	214.988 ms	45.764 ms	4.70×	CSR SIMD (41.3× faster)
pkustk14	4.023 ms	2.433 ms	1.65×	26.088 ms	17.464 ms	1.49×	CSR SIMD (7.2× faster)
roadNet-CA	2.741 ms	3.020 ms	0.91× (Slowdown)	4.741 ms	3.360 ms	1.41×	CSR Scalar (1.2× faster)

Supporting Plot:



Key Findings

CSR SIMD regresses on short rows but wins on dense rows. Rows shorter than 8 nonzeros fail the 8-lane loop check and fall back to the scalar tail while still incurring register setup and horizontal reduction overhead. But on dense rows, vector iterations per row amortize horizontal reduction, yiedling speedups.

ELL SIMD is able to keep the 8 SIMD lanes full regardless of row length, achieving speedups on all matrices.

C. CSR vs. ELL Format Comparison Across Matrices and Schedules

Matrix	Max K	Avg μ	Padding	CSR (8-th)	ELL (8-th)	CSR SIMD	ELL SIMD	Overall Winner	Speedup Margin
bfly	4	4.0	0.0%	0.050 ms	0.039 ms	0.050 ms	0.026 ms	ELL	1.93× faster (SIMD)
D6-6	6	1.2	79.7%	0.130 ms	0.092 ms	0.145 ms	0.058 ms	ELL	2.52× faster (SIMD)
dictionary28	38	3.4	91.1%	0.112 ms	0.361 ms	0.123 ms	0.178 ms	CSR	3.22× faster (OpenMP) / 1.44× (SIMD)
Ga3As3H12	1,622	97.3	94.0%	1.906 ms	214.35 ms	1.107 ms	45.76 ms	CSR	112.4× faster (OpenMP) / 41.4× (SIMD)
pkustk14	333	97.7	70.7%	3.964 ms	25.96 ms	2.433 ms	17.46 ms	CSR	6.55× faster (OpenMP) / 7.18× (SIMD)
roadNet-CA	12	2.8	76.6%	2.734 ms	4.776 ms	3.020 ms	3.360 ms	CSR	1.75× faster (OpenMP) / 1.11× (SIMD)

Per-Matrix Breakdown & Rationale:

1. **bfly** — Column-Major ELL (1.93× faster):
- **Why:** Every row contains exactly 4 nonzeros ($K = \mu = 4.0$), resulting in **0.0% zero padding**. ELL has zero wasted memory traffic and eliminates CSR's `row_ptr` array entirely (saving 11% storage). In SIMD, Column-Major ELL loads 8 rows at a time with 100% vector lane utilization, whereas CSR's row length of 4 is shorter than the 8-lane SIMD width, forcing scalar fallback and horizontal reduction overhead.
2. **D6-6** — Column-Major ELL (2.52× faster):

- **Why:** Although padding is 79.7%, the maximum row length is very small ($K = 6$), keeping total ELL memory compact (5.79 MB, fitting within cache). Column-major layout enables regular, contiguous memory streaming across 8 rows, achieving a massive 159.1 GB/s memory throughput. In contrast, CSR's tiny average row length ($\mu = 1.2$) incurs severe branch checks, constant loop restarts, and scalar reduction penalties in SIMD.

3. **dictionary28** — CSR (3.22× faster in OpenMP, 1.44× in SIMD):

- **Why:** Outlier rows reach $K = 38$ while average row length is only $\mu = 3.4$, causing **91.1% zero padding** and expanding memory by **9.79×** (16.01 MB vs 1.64 MB). Streaming 14.4 MB of padded zeros across the memory bus severely throttles ELL, allowing CSR's compact, dense representation to win decisively.

4. **Ga3As3H12** — CSR (112.4× faster in OpenMP, 41.35× in SIMD):

- **Why:** A handful of dense orbital rows of length $K = 1,622$ (vs. $\mu = 97.3$) forces ELL to allocate 796.06 MB (**16.58× memory expansion, 94.0% padding waste**). Moving hundreds of megabytes of padding zeros completely saturates DRAM bandwidth. CSR computes strictly over actual nonzeros, and its long rows allow CSR SIMD along-row vectorization to achieve 1.73× speedup without streaming any padding zeros.
- **Across Schedules:** CSR outperforms ELL under all schedules: by **34.1×** under **dynamic,1** (5.96 ms vs 203.15 ms), **154.9×** under **dynamic,64** (1.24 ms vs 191.92 ms), and **261.9×** under **nnz-static** (1.33 ms vs 347.26 ms).

5. **pkustk14** — CSR (6.55× faster in OpenMP, 7.18× in SIMD):

- **Why:** ELL memory footprint balloons to 404.73 MB vs. 119.30 MB for CSR (**70.7% zero padding**). Because rows are long ($\mu \approx 98$), CSR along-row SIMD achieves 1.65× speedup with full vector utilization on true nonzeros, avoiding the 285 MB of padding traffic that throttles ELL.
- **Across Schedules:** CSR consistently outperforms ELL across all policies: **row-static** (6.40×), **row-dynamic,16** (6.41×), **row-dynamic,64** (7.32×), **row-guided** (7.32×), and **nnz-static** (7.69×).

6. **roadNet-CA** — CSR (1.75× faster in OpenMP, 1.11× in SIMD):

- **Why:** With nearly 2 million rows ($m = 1,971,281$), even a moderate padding of 76.6% ($K = 12, \mu = 2.8$) inflates memory from 52.15 MB to 189.24 MB (+137 MB of padding zeros per iteration). The aggregate DRAM bandwidth penalty of streaming this excess data outweighs the benefit of ELL across-row vectorization.

E. Combined Empirical Roofline Analysis

Arithmetic Intensity: $AI_{\text{CSR}} = \frac{2 \cdot \text{nnz}}{\text{Bytes}} \leq \frac{2 \cdot \text{nnz}}{12 \cdot \text{nnz}} \approx 0.167 \text{ FLOP/byte}$ (4B vals + 4B col_idx + 4B $x[\text{col}]$).

Machine Ceilings: $P_{\text{peak}} = 537.6 \text{ GFLOP/s}$, $B_{\text{peak}} \approx 200.0 \text{ GB/s}$.

Machine Ridge Point: $I^* = \frac{P_{\text{peak}}}{B_{\text{peak}}} = \frac{537.6}{200.0} = 2.688 \text{ FLOP/byte}$.

Max Theoretical Performance: $P_{\max} = AI \times B_{\text{peak}} = 0.167 \times 200.0 = 33.4 \text{ GFLOP/s}$ (only **6.2% of peak compute**).

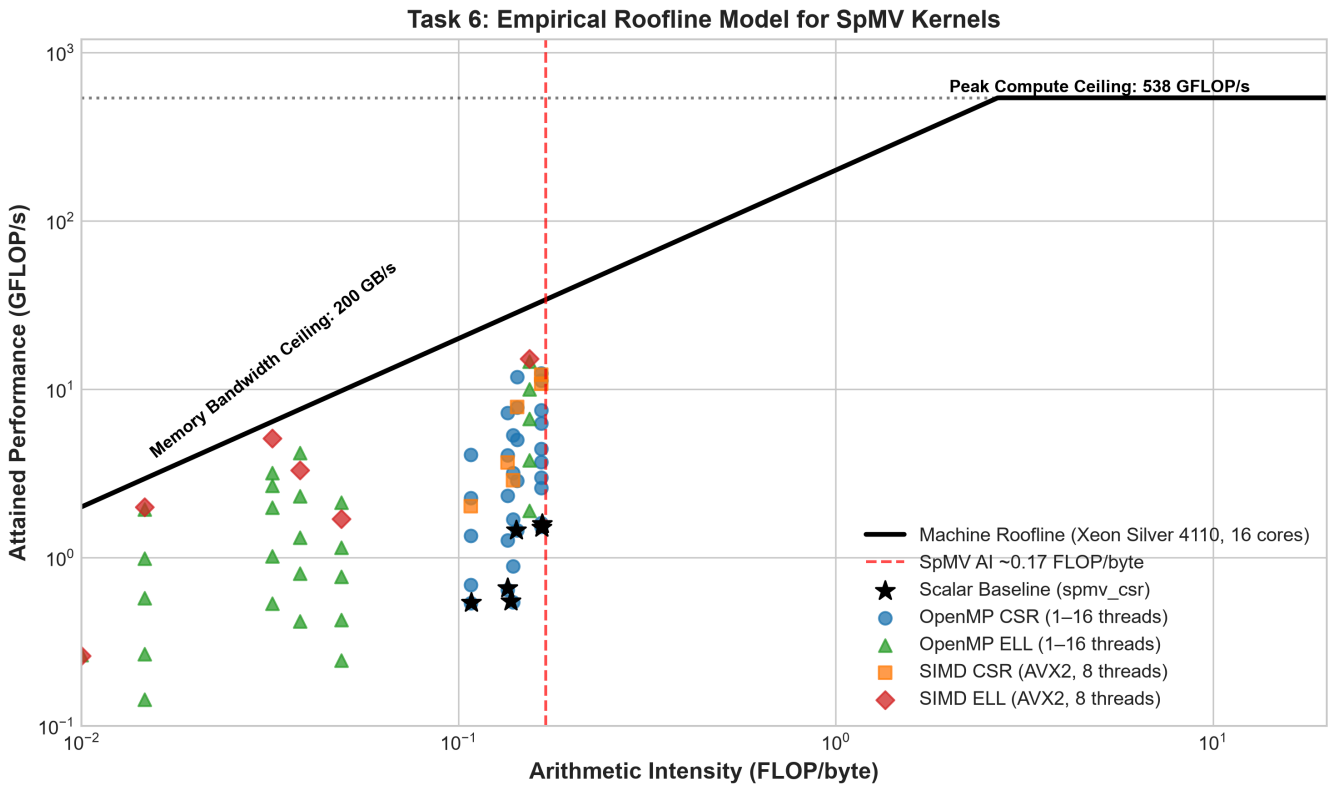
Machine Peak:

- **Peak Memory Bandwidth** = $\sim 200.0 \text{ GB/s}$
- **Peak Compute** = 537.6 GFLOP/s

Achieved Bandwidth:

- **ELL SIMD:** 159.1 GB/s
- **CSR:** 73.7 GB/s

Supporting Plot:



SpMV is fundamentally memory-bound because its low arithmetic intensity ($AI \leq 0.167 \text{ FLOP/byte}$) is over $16\times$ below the machine ridge point ($I^* = 2.688 \text{ FLOP/byte}$), capping theoretical performance at 33.4 GFLOP/s —just 6.2% of the dual Xeon Silver 4110’s 537.6 GFLOP/s compute peak. Because performance is restricted by the $\approx 200 \text{ GB/s}$ memory bandwidth ceiling, adding compute capacity through threading or vectorization yields severe diminishing returns. In OpenMP, all 16 cores contend for the same shared DDR4 channels, capping 16-thread speedup at only $7.34\times$ – $7.78\times$ ($E \approx 46\%$ – 49% on [Ga3As3H12](#) and [pkustk14](#)) as memory traffic saturates the bus at up to 75.19 GB/s . In AVX2 SIMD, despite an $8\times$ vector lane width, CSR speedups cap at $1.65\times$ – $1.73\times$ on long rows and drop to $0.90\times$ – $0.91\times$ on short rows ([D6-6](#), [dictionary28](#), [roadNet-CA](#)) due to horizontal reductions and deserialized gathers ([_mm256_i32gather_ps](#)). Furthermore, while ELL SIMD streams at 159.1 GB/s (79.6% of STREAM peak) on [D6-6](#), on [Ga3As3H12](#) 94.0% of that traffic is wasted on zero padding, leaving CSR SIMD $41.3\times$ faster. Ultimately, because ALUs are permanently starved for bytes, wider vectors and additional threads merely increase idle cycles waiting on DRAM.

7. Write-Up: Design Decisions, Anomalies & Conclusions

A. Design Decisions

1. **Precomputed Setup (`spmv_omp_setup`):** Structural operations—CSR-to-ELL conversion and binary search for `nnz-static` thread row boundaries—are precalculated once during setup, ensuring zero memory allocation or partitioning overhead during timed SpMV iterations.
 2. **Column-Major ELL Layout:** Mapping entry $(i, k) \rightarrow k \cdot m + i$ places slot k of 8 consecutive rows contiguously in physical RAM, enabling single 256-bit SIMD loads (`_mm256_loadu_ps`) and eliminating strided memory gathers for matrix values.
 3. **Whole-Row Thread Assignment:** Threads always own whole rows in both row- and nonzero-partitioning, eliminating inter-thread race conditions on $y[\text{row}]$ and avoiding locks or atomic reductions.
-

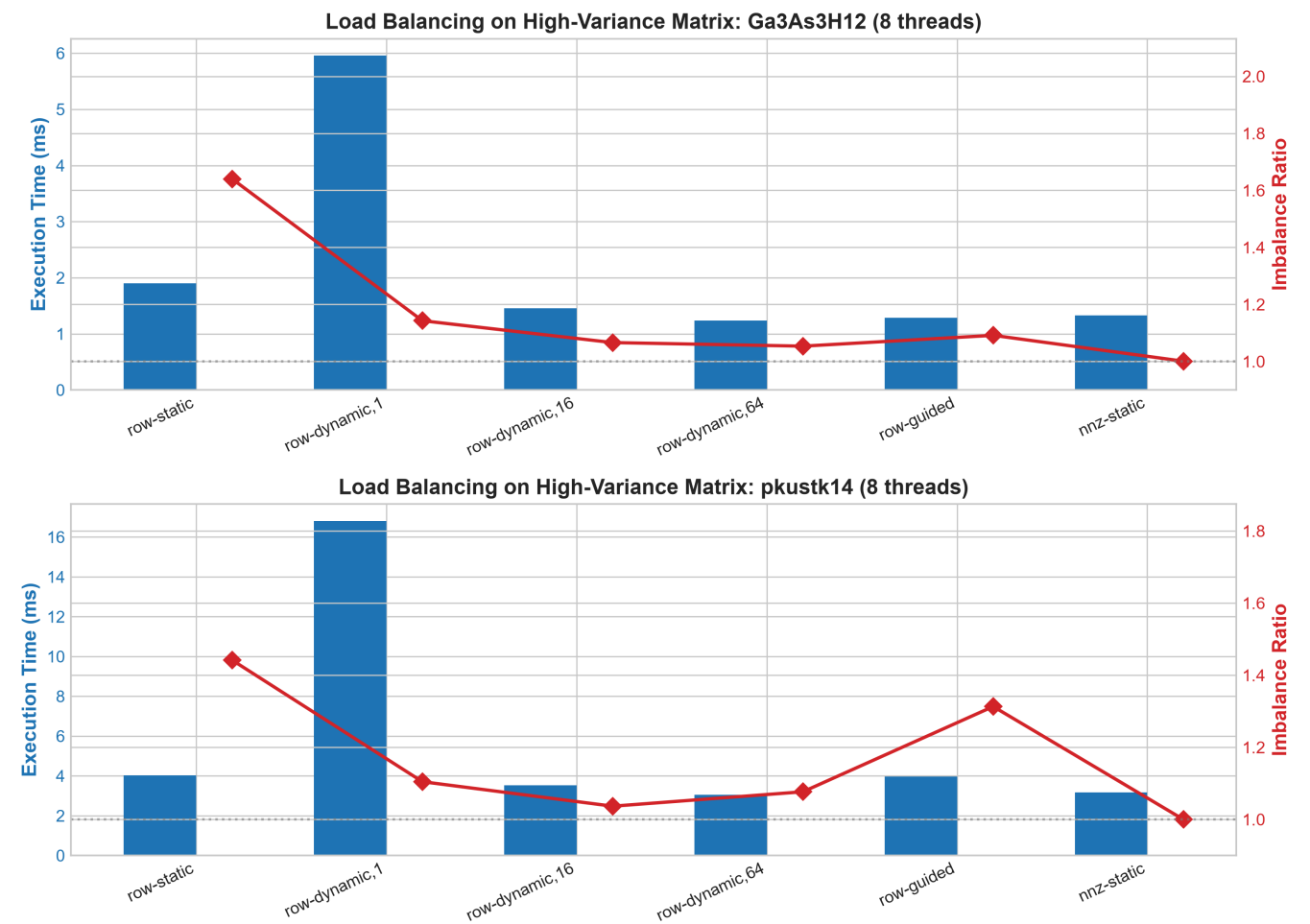
B. Format Trade-Off: CSR vs. Column-Major ELL

At a high level, the trade-off is **storage regularity vs. memory waste**:

- **CSR** is strictly compact (only nonzeros are stored; 0% padding), minimizing memory bus traffic. However, its irregular row lengths hinder vectorization and create workload imbalance across threads.
 - **ELL** transforms sparse storage into a uniform dense $m \times K$ rectangle, unlocking regular memory streaming and efficient vectorization across rows. However, when the maximum row length far exceeds the average ($K \gg \mu$), ELL wastes memory by allocating up to 94% zero padding, saturating DRAM bandwidth with useless zeros.
 - **Decision Rule:** If $\frac{K}{\mu} \leq 1.5$ (e.g., `bfly`), use **Column-Major ELL SIMD**; if $\frac{K}{\mu} > 1.5$, **CSR** decisively wins.
-

C. Load-Balancing Results (8 Threads)

Supporting Plot:



Matrix	Policy	Runtime (ms)	Imbalance Ratio	Attained Bandwidth	Systems Assessment
Ga3As3H12	row-static	1.899 ms	1.64×	37.99 GB/s	Naive baseline; severe thread tail stall
	row-dynamic,1	5.960 ms	1.14×	12.10 GB/s	3.14× slowdown: Central work-queue lock contention
	row-dynamic,16	1.457 ms	1.07×	49.51 GB/s	Coarse chunk amortizes work-queue locks
	row-dynamic,64	1.239 ms	1.05×	58.22 GB/s	Fastest dynamic: Preserves spatial cache prefetching
	row-guided	1.286 ms	1.09×	56.12 GB/s	Smooth decay avoids fine-grained lock contention
	nnz-static	1.326 ms	1.00×	54.41 GB/s	Ideal 1.00 balance; 30.2% faster than static (0 locks)
pkustk14	row-static	4.030 ms	1.44×	44.47 GB/s	Irregular dense structural blocks stall threads

Matrix	Policy	Runtime (ms)	Imbalance Ratio	Attained Bandwidth	Systems Assessment
	row-dynamic,1	16.811 ms	1.10×	10.66 GB/s	4.17× catastrophe: Atomic mutex thrashing
	row-dynamic,16	3.531 ms	1.04×	50.76 GB/s	Substantial runtime recovery over chunk=1
	row-dynamic,64	3.054 ms	1.08×	58.70 GB/s	Fastest dynamic (24.2% faster than static)
	row-guided	3.972 ms	1.31×	45.13 GB/s	Large initial chunks leave residual tail imbalance
	nnz-static	3.163 ms	1.00×	56.66 GB/s	Ideal 1.00 balance; 21.5% faster than static (0 locks)

- **Key Takeaway (Imbalance $\neq$ Speed):** Balancing work algorithmically can backfire. Setting `schedule(dynamic,1)` flattens imbalance to $\approx 1.10\times$, but causes a **4.17×** wall-clock slowdown due to atomic work-queue contention and cache thrashing. In contrast, `nnz-static` achieves perfect $1.00\times$ balance with zero runtime synchronization.

D. Where SIMD Did / Did Not Help & Why (Memory-Bound Discussion)

- **Where SIMD Helped:**
 - **Long rows in CSR (Ga3As3H12: 1.73×**, **pkustk14: 1.65×**): Rows with $\mu \approx 98$ execute multiple full 8-lane AVX2 iterations, easily amortizing the single horizontal reduction at the end of the row.
 - **Column-Major ELL Across Rows:** SIMD achieved speedups on all matrices ($1.41\times$ to $4.70\times$) because grouping 8 rows guarantees 100% lane utilization without horizontal reductions.
- **Where SIMD Did Not Help / Regressed:**
 - **Short rows in CSR (D6-6: 0.90×**, **dictionary28: 0.91×**, **roadNet-CA: 0.91×**): When average row length is smaller than the 8-lane SIMD width ($\mu < 8$), the vector loop is skipped. The processor incurs loop setup, branch checks, and scalar fallback penalties, resulting in a net slowdown.
 - **High-variance ELL:** On `Ga3As3H12`, despite a $4.70\times$ SIMD speedup over scalar ELL, ELL SIMD was still $41.3\times$ slower than CSR SIMD because 94% of its memory traffic consisted of empty zero padding.
- **Why (The Memory-Bound Root Cause):** SpMV's arithmetic intensity ($AI \leq 0.17$ FLOP/byte) is $16\times$ below the machine ridge point (2.69 FLOP/byte). Vectorization only accelerates ALU throughput, but cannot speed up DRAM fetch latency or bus bandwidth. Once data arrives, FMA finishes in 1 cycle, leaving the ALUs stalled waiting on DRAM.

E. Per-Matrix Behavior Summary

1. **bf1y (Winner: ELL SIMD, 1.93×** faster): Perfectly uniform ($K = \mu = 4.0$, 0% padding). ELL eliminates the `row_ptr` array and streams 8 rows at a time with zero waste.

2. **D6-6 (Winner: ELL SIMD, $2.52\times$ faster):** Compact footprint (5.79 MB, fits in LLC). Despite 79.7% padding, small $K = 6$ enables contiguous streaming at 159.1 GB/s, avoiding CSR's short-row penalties ($\mu = 1.2$).
 3. **dictionary28 (Winner: CSR, $3.22\times$ faster):** High variance ($K = 38$, $\mu = 3.4$) causes 91.1% padding and balloons ELL memory by $9.8\times$. CSR's compact layout easily wins.
 4. **Ga3As3H12 (Winner: CSR, $41.3\times$ faster in SIMD, $112.4\times$ in OpenMP):** Outlier rows ($K = 1$, 622 vs. $\mu = 97.3$) cause 94.0% zero padding (796 MB vs. 48 MB). Moving hundreds of megabytes of zeros throttles the memory bus.
 5. **pkustk14 (Winner: CSR, $7.18\times$ faster in SIMD):** 70.7% ELL padding penalty (405 MB vs. 119 MB). CSR along-row SIMD achieves $1.65\times$ speedup over true nonzeros without streaming padding zeros.
 6. **roadNet-CA (Winner: CSR, $1.75\times$ faster):** Large row count ($m \approx 2\text{M}$) amplifies even moderate padding (76.6%) into +137 MB of redundant memory traffic per iteration.
-

F. Anomalies & Takeaways

1. **Fine-Grained Dynamic Contention:** `schedule(dynamic,1)` achieves near-perfect load balance ($1.10\times$) but causes a $4.17\times$ slowdown due to atomic work-queue mutex contention and cache line thrashing.
2. **The Short-Row Vector Penalty:** Vectorizing along short rows ($\mu < 8$) introduces loop setup and horizontal reduction overhead that exceeds scalar execution, causing net regressions.
3. **The ELL Memory Paradox:** ELL SIMD achieved near-peak machine memory bandwidth (159.1 GB/s, 79.6% of STREAM peak), yet lost to CSR by up to $41.3\times$ because the memory bus was flooded with uninformative padding zeros.